Centrifuge Audit Report

Prepared by Burra Security

Version 1.0

Researchers

LonelySloth

kiki_dev

taridoku

April 27, 2026

Contents

1 About Burra Security **2**

2 Disclaimer **2**

3 Risk Classification **2**

4 Protocol Summary **2**

5 Audit Scope **2**

6 Executive Summary **2**

7 Findings **5**

7.1 Medium Risk . . . 5

7.1.1 OnOffRamp.trustedCall(Withdraw) breaks withdrawal accounting symmetry . . . 5

7.1.2 PM guards fail open on zero-valued price state . . . 6

7.2 Low Risk . . . 8

7.2.1 SlippageGuard wipes period on config change . . . 8

7.2.2 Command pointing to unpinned state allows arbitrary function calls . . . 8

7.2.3 OnchainPM has no ETH sweep mechanism, enabling cross-strategist ETH spending . . . 9

7.2.4 Unpinned onchainPM in FlashLoanHelper enables Merkle policy bypass via malicious pool re-entry . . . 9

7.2.5 SlippageGuard cumulative period loss and per-trade maxSlippageBps can be bypassed with deposits by other parties . . . 13

7.2.6 ScriptHelpers: decimal conversion has fixed rounding down behavior. . . . 13

7.3 Informational . . . 15

7.3.1 SlippageGuard can be DoS'd by other calls in same transaction . . . 15

7.3.2 Lack of callback+command ordering guarantee in OnchainPM . . . 15

7.3.3 6909 transferFrom reduces allowance when approval is set to max . . . 16

7.3.4 Admin paths emit after no-op storage write . . . 16

7.3.5 ScriptHelpers: `abs` unexpectedly revert on minimum valid signed integer. . . . 16

7.3.6 ScriptHelpers: addBps unnecessarily limits addition to 100%. . . . 17

7.3.7 Lack of validation of non-empty assets array in SlippageGuard . . . 17

7.3.8 State bitmap isn't checked to contain only bits for valid state indexes . . . 17

7.3.9 Lack of index range validation in TransientArrayLib . . . 18

1 About Burra Security

Burra Security offers security auditing and advisory services with a special focus on cross-chain and interoperability protocols and their integrations. Learn about us at burrasec.com.

2 Disclaimer

A smart contract security review can never verify the complete absence of vulnerabilities. This is a time, resource, and expertise-bound effort where we try to find as many vulnerabilities as possible. We can not guarantee 100% security after the review or even if the review will find any vulnerabilities. Subsequent security reviews, bug bounty programs, and on-chain monitoring are recommended.

3 Risk Classification

	Impact: High	**Impact: Medium**	**Impact: Low**
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

4 Protocol Summary

PR 167 introduces `OnchainPM`, a Weiroll based execution engine that authorizes whole scripts via Merkle proofs, with a `stateBitmap` that pins governance approved state elements while allowing strategist supplied runtime inputs. It adds callback support through `executeCallback()` and a `FlashLoanHelper` pattern so flash loan driven strategies can safely re enter execution using pre committed callback hashes and expected callers. It also introduces an ERC 6909 `AccountingToken` and updates `OnOffRamp` and `OracleValuation` to improve accounting and valuation for in flight assets, including cross chain bridging receipts and spoke side price updates, with additional helper and guard contracts for slippage bounds, approval cleanup, and circuit breaking.

5 Audit Scope

Audit scope includes `https://github.com/centrifuge/protocol-internal/pull/167` and the following files:

- src-ir/OnchainPM.sol
- src/managers/spoke/guards/*.sol
- src/managers/spoke/AccountingToken.sol
- src/managers/spoke/ScriptHelpers.sol
- src/managers/spoke/FlashLoanHelper.sol
- src/managers/spoke/OnOffRamp.sol (diff only)
- src/misc/libraries/TransientArrayLib.sol (diff only)
- src/valuations/OracleValuation.sol (diff only)

6 Executive Summary

Over the course of 4 days, the Burra Security team conducted an audit on the Centrifuge smart contracts provided by Centrifuge. In this period, a total of 17 issues were found.

Summary of the audit findings and any additional executive comments.

Summary

Project Name	Centrifuge
Repository	protocol-internal
Commit	160b2611ebcc...
Audit Timeline	April 6th - April 9th
Methods	Manual Review

Issues Found

Critical Risk	0
High Risk	0
Medium Risk	2
Low Risk	6
Informational	9
Gas Optimizations	0
Total Issues	17

Summary of Findings

[M-1] OnOffRamp.trustedCall(Withdraw) breaks withdrawal accounting symmetry	Resolved
[M-2] PM guards fail open on zero-valued price state	Resolved
[L-1] SlippageGuard wipes period on config change	Resolved
[L-2] Command pointing to unpinned state allows arbitrary function calls	Acknowledged
[L-3] OnchainPM has no ETH sweep mechanism, enabling cross-strategist ETH spending	Resolved
[L-4] Unpinned onchainPM in FlashLoanHelper enables Merkle policy bypass via malicious pool re-entry	Resolved
[L-5] SlippageGuard cumulative period loss and per-trade maxSlippageBps can be bypassed with deposits by other parties	Acknowledged
[L-6] ScriptHelpers: decimal conversion has fixed rounding down behavior.	Resolved
[I-1] SlippageGuard can be DoS'd by other calls in same transaction	Resolved
[I-2] Lack of callback+command ordering guarantee in OnchainPM	Resolved
[I-3] 6909 transferFrom reduces allowance when approval is set to max	Resolved
[I-4] Admin paths emit after no-op storage write	Acknowledged

[I-5] ScriptHelpers: `abs` unexpectedly revert on minimum valid signed integer.	Resolved
[I-6] ScriptHelpers: addBps unnecessarily limits addition to 100%.	Resolved
[I-7] Lack of validation of non-empty assets array in SlippageGuard	Resolved
[I-8] State bitmap isn't checked to contain only bits for valid state indexes	Resolved
[I-9] Lack of index range validation in TransientArrayLib	Resolved

7 Findings

7.1 Medium Risk

7.1.1 OnOffRamp.trustedCall(Withdraw) breaks withdrawal accounting symmetry

Target: OnOffRamp.sol

Severity:

- Impact: Medium
- Likelihood: Medium

Description: `trustedCall(Withdraw)` performs the same class of withdrawal operation while omitting the accounting token mint.

`OnOffRamp` is documented as a manager with accounting-token support:

- deposits mint a liability accounting token
- withdrawals mint a non-liability accounting token as a receipt for the withdrawn asset

That model is implemented in the relayer-driven `withdraw()` path which performs a full `BalanceSheet` withdrawal and then mints/deposits the non-liability accounting token.

By contrast, the `TrustedCall.Withdraw` branch also performs a full `BalanceSheet` withdrawal, but stops there:

```
} else if (kind == TrustedCall.Withdraw) {
   // ...
   balanceSheet.withdraw(poolId, scId, asset, 0, receiver, amount, WithdrawMode.Full);
   emit TrustedWithdraw(asset, amount, receiver);
}
```

This is important because `WithdrawMode.Full` is the standard OnOffRamp withdrawal mode: escrow accounting, Hub queue update, and transfer. So `trustedCall(Withdraw)` is not bypassing the withdrawal model altogether; it is opting into the same real-asset withdrawal semantics as `withdraw()`.

Compare with the relayer `withdraw()` path:

```
function withdraw(...) external {
   // ...
   balanceSheet.withdraw(poolId, scId, asset, 0, receiver, amount, WithdrawMode.Full);

   uint256 accTokenId = accountingToken.toTokenId(poolId, asset, false);
   accountingToken.mint(address(this), accTokenId, amount, scId);
   accountingToken.approve(address(balanceSheet), accTokenId, amount);
   balanceSheet.deposit(poolId, scId, address(accountingToken), accTokenId, amount);
}
```

`trustedCall(Withdraw)` serves as a governance-operated fallback for the same offramp operation when the normal relayer path is unavailable. Under that interpretation, it should preserve the same accounting semantics as the ordinary withdrawal path.

As written, every successful trusted withdrawal creates an accounting asymmetry:

- the real asset leaves escrow through the full withdrawal path
- the withdrawal is queued / accounted on the BalanceSheet side
- but no non-liability receipt token is minted to mirror the outflow

So the trusted path breaks the withdrawal accounting symmetry that the contract header and the standard `withdraw()` implementation otherwise establish.

Recommended Mitigation: Because `trustedCall(Withdraw)` uses the same `WithdrawMode.Full` withdrawal semantics and appears to act as a privileged fallback for the standard offramp flow, it should mirror the receipt-token minting performed by `withdraw()`:

```
} else if (kind == TrustedCall.Withdraw) {
   (, uint128 assetId, uint128 amount, bytes32 receiverAddress) =
      abi.decode(payload, (uint8, uint128, uint128, bytes32));
   (address asset,) = balanceSheet.spoke().idToAsset(AssetId.wrap(assetId));
   address receiver = receiverAddress.toAddress();

   require(offramp[asset][receiver], InvalidOfframpDestination());
   balanceSheet.withdraw(poolId, scId, asset, 0, receiver, amount, WithdrawMode.Full);

   uint256 accTokenId = accountingToken.toTokenId(poolId, asset, false);
   accountingToken.mint(address(this), accTokenId, amount, scId);
   accountingToken.approve(address(balanceSheet), accTokenId, amount);
   balanceSheet.deposit(poolId, scId, address(accountingToken), accTokenId, amount);

   emit TrustedWithdraw(asset, amount, receiver);
}
```

Client: Fixed in centrifuge/protocol-internal#175.

BurraSec: Fix looks good. Now routes through `_withdraw()`

7.1.2 PM guards fail open on zero-valued price state

Target: CircuitBreakerGuard.sol, SlippageGuard.sol

Severity:

- Impact: Medium
- Likelihood: Medium

Description: The codebase supports zero-valued price updates. There's no guard in `OracleValuation: _setPrice`:

```
   function _setPrice(PoolId poolId, ShareClassId scId, AssetId assetId, D18 newPrice) internal {
      pricePoolPerAsset[poolId][scId][assetId] = Price(newPrice, true);
      hub.updateHoldingValue(poolId, scId, assetId);
      emit UpdatePrice(poolId, scId, assetId, newPrice);
   }
```

and unit tests in `OracleValuation.t.sol:testSetPriceZeroPrice` also support the conclusion. However, the issue is that the PM guard layer treats that 0 as a harmless state and, as a result, can stop enforcing risk checks if price integrity fails.

`CircuitBreakerGuard.delta()` is intended to bound per-update value changes such as price updates. Per the NatSpec, its `currentValue` parameter is dynamic and is read from on-chain state via prior weiroll commands. However, when `currentValue == 0`, the function silently returns without enforcing any bound:

```
if (currentValue == 0) return;
```

So, the breaker is disabled at a moment where it should be halting operations. This highlights two different cases:

- legitimate initialization / reset flows where the old value happens to be zero
- fault states where a zero value reflects broken pricing, misconfiguration, stale state, or a deliberately injected zero In both cases, the check is waived.

`SlippageGuard` has a similar failure mode. It measures loss by converting balance deltas into pool units using the current spoke-side asset price. Where that price is zero, then even large real balance losses evaluate to zero pool-unit loss:

- `withdrawn` can evaluate to `0`
- `deposited` can evaluate to `0`
- `loss` becomes `0`
- neither the per-script slippage check nor the cumulative period-loss tracker fires

Combined, loss accounting goes blind.

Recommended Mitigation: Make `CircuitBreakerGuard` and `SlippageGuard` explicitly reject or specially handle zero-valued inputs:

```
require(currentValue != 0, ZeroAnchor());
...
```

Client: Fixed in centrifuge/protocol-internal#175.

BurraSec: Fix looks good. Reverts for 0 price on loss.

7.2 Low Risk

7.2.1 SlippageGuard wipes period on config change

Target: SlippageGuard.sol

Severity:

- Impact: High
- Likelihood: Low

Description: SlippageGuard `trustedCall` writes in a new `maxPeriodLoss` and `periodDuration`, then deletes period for that pool and share class. Resulting in the rolling cumulative loss limit enforced in close through the `_trackPeriodLoss` function to not continue across a configuration change. This allows for total loss to exceed the new period loss. Take for example:

- A max loss of 100,000 in a 24 hour period
- The current lost amount is 90,000 at the 6 hour mark
- A `trustedCall` updates the max `maxPeriodLoss` and `periodDuration` to be 50,000 in a 24 hour period
- current period values are now set to 0 allowing for another 50,000 in losses to occur
- Net losses is now 140,000 in less than 24 hours. Larger than both the previous and current configurations.

If the old period data is not deleted on config changes then losses will not exceed the current max and will naturally rollover and clear the previous period data when the `periodDuration` is met.

Recommended Mitigation: Considering preserving `cumulativeLoss` and `periodStart` when calling `trustedCall` in the `SlippageGuard` contract.

Client: Fixed in centrifuge/protocol-internal#175.

BurraSec: Fix looks good. `delete` removed and config is preserved.

7.2.2 Command pointing to unpinned state allows arbitrary function calls

Target: OnchainPM.sol

Severity:

- Impact: High
- Likelihood: Low

Description: `OnchainPM` binds execution to a Merkle leaf through the `computeScriptHash` function. State slots whose bitmap bit is clear do not contribute their bytes to the script hash. This means that if a `command` points to a state slot that is not pinned and the weiroll command uses the `DATA` flag, the strategist can populate those bytes with anything, including the four-byte selector, allowing them to access any unintended function at the target address with any parameters they see fit.

In instances where it is intended that `transfer` is going to be called but because the function is unpinned, instead the strategist calls `approve`. And since approval wasn't expected in the workflow the required guards were not added and the existing circuit breaker guards pass as no funds have been moved yet. Now the strategist can just transfer themselves any and all funds that they wish after the workflow completes.

Recommended Mitigation: Verify on-chain that all commands which use flag data point to a pinned state index.

Client: This is an admin error, providing insecure input (DATA flag)

BurraSec: Acknowledged

7.2.3 OnchainPM has no ETH sweep mechanism, enabling cross-strategist ETH spending

Target: OnchainPM.sol

Severity:

- Impact: Medium
- Likelihood: Low

Description: `OnchainPM.execute()` is payable, allowing strategists to send ETH for use in VALUECALL commands (e.g., cross-chain gas payments, DEX interactions). However, OnchainPM has no mechanism to return excess ETH to the strategist after script execution.

The vulnerable case is direct `execute{value: ...}()` usage. If a strategist sends more ETH than the approved weiroll script actually consumes through VALUECALL, the unused remainder stays on `OnchainPM`.

Since multiple strategists are supported via `mapping(address strategist => bytes32 root) public policy`, and there is no per-strategist ETH accounting, any strategist with an approved script containing a VALUECALL command can spend the entire ETH balance of OnchainPM, including ETH deposited by other strategists.

Recommended Mitigation:

1. Automatically refund remaining ETH to the caller in `execute()`:

```
function execute(...) external payable {
   uint256 balanceBefore = address(this).balance - msg.value;
   // ... existing logic ...
      callbackIdx = 0;
// -----------------After callback check --------------
   uint256 excess = address(this).balance - balanceBefore;
   if (excess > 0) {
      (bool success,) = payable(msgSender()).call{value: excess}("");
      require(success);
   }
// -----------------Before active strategist clear----------
      activeStrategist = address(0);
   // rest of logic
}
```

The refund must be placed before active strategist is cleared and after the callback check as a reentrancy guard.

2. Add a permissioned sweep function accessible via `trustedCall` for governance-controlled ETH recovery.

Client: Fixed in centrifuge/protocol-internal#175.

BurraSec: Fix looks good. Refunds excess over the pre-existing balance.

7.2.4 Unpinned onchainPM in FlashLoanHelper enables Merkle policy bypass via malicious pool re-entry

Target: FlashLoanHelper.sol, OnchainPM.sol

Severity:

- Impact: High
- Likelihood: Low

Description: `FlashLoanHelper.requestFlashLoan` is permissionless and transfers flash-loaned tokens to whatever address is passed as `onchainPM` (line 50), then calls `executeCallback` on it (line 54). The NatSpec warns that `pool` must be bitmap-locked but is silent on `onchainPM`.

In practice, the protocol permits asset originators to manage a family of pools (different risk profiles, funding structures). These pools each have their `poolId`, and by extension will have their `onchainPM`. Pinning `onchainPM` in such cases would require a separate script hash per pool for every flash loan script, creating a case where the

approval set becomes much larger and harder to manage. Practically, to operate at scale, for identical operations across all their pools, a shared strategist + policy is logical, in which case, a free `onchainPM` slot is apt.

A strategist can substitute a controlled contract as `onchainPM`, receiving the full flash loan amount with unconstrained execution - no weiroll VM, no Merkle policy, no bitmap constraints. There are three main checks in `OnchainPM.executeCallback` that appear to prevent this:

1. **Callback caller check**:

```
require(msg.sender == TransientArrayLib.atAddress(CALLBACK_CALLERS_SLOT, idx))
```

only the declared caller (FlashLoanHelper) can invoke `executeCallback`. The caller address is committed in the script hash, so the strategist cannot change it.

2. **Script hash check**:

```
require(scriptHash == expected)
```

the inner callback script's commands and bitmap-locked state must match the hash committed at script approval. The strategist cannot run a different inner script.

3. **Unconsumed callbacks check**:

```
require(callbackIdx == TransientArrayLib.length(CALLBACK_HASHES_SLOT))
```

all declared callback slots must be consumed before `execute()` returns. If the attacker redirects the flash loan but never consumes the slot on the real OnchainPM, the transaction reverts.

Together these appear to guarantee that even with an unpinned `onchainPM`, the flash-loaned capital can only be used through the governance-approved inner callback script on the real OnchainPM. However, all three walls are bypassable via a single re-entry pattern using a malicious pool. Consider this:

1. `OnchainPM.execute()` opens the callback window and pre-commits `FlashLoanHelper` as the allowed callback caller.

```
activeStrategist = msgSender();
TransientArrayLib.push(CALLBACK_HASHES_SLOT, callbacks[i].hash);
TransientArrayLib.push(CALLBACK_CALLERS_SLOT, callbacks[i].caller);
```

At this point the real OnchainPM is in an active execution context, but no callback has been consumed yet.

2. The outer approved weiroll leaf calls `FlashLoanHelper.requestFlashLoan(...)` with a free `onchainPM` slot pointing at the strategist's controlled contract.

```
_pool = address(pool);
_onchainPM = address(onchainPM);
pool.flashLoanSimple(address(this), token, amount, callbackData, 0);
```

This is the first exploited boundary: `requestFlashLoan` is permissionless and trusts caller-supplied `onchainPM` without validating that it is the real pool-specific OnchainPM.

3. During the legitimate Aave callback, `FlashLoanHelper.executeOperation()` transfers the full flash-loaned amount to the attacker-chosen `onchainPM` before any real callback slot is consumed.

```
address onchainPM = _onchainPM;
asset.safeTransfer(onchainPM, amount);
IOnchainPM(onchainPM).executeCallback(commands, state, stateBitmap);
```

At this point, the strategist now has temporary custody of the entire flash loan and arbitrary execution in their own contract rather than inside the weiroll VM.

4. From that unconstrained callback, the strategist re-enters `FlashLoanHelper.requestFlashLoan(...)` using another controlled contract as pool and sets `onchainPM` back to the real OnchainPM.

```
helper.requestFlashLoan(
   IAaveV3Pool(address(maliciousPool)),
   token,
   0,
   realOnchainPM,
   callbackData2
);
```

This is the second exploited boundary: the nested call overwrites `_pool` and `_onchainPM` for the inner flash-loan session, allowing them to manufacture a clean callback path to the real OnchainPM while keeping the original borrowed funds outside it.

5. The pool immediately calls back into `FlashLoanHelper.executeOperation()`, which now accepts the inner call because the nested `requestFlashLoan` set `_pool` to the malicious pool.

```
require(msg.sender == _pool, NotPool());
require(_onchainPM != address(0), NotActive());
require(initiator == address(this), NotInitiator());
```

Since the inner amount can be `0`, no fresh capital is needed for this second call. Its sole purpose is to consume the legitimate callback slot on the real OnchainPM.

6. `OnchainPM.executeCallback()` on the real OnchainPM accepts the inner callback, because all three checks are satisfied through the helper mediated re entry path.

```
require(msg.sender == TransientArrayLib.atAddress(CALLBACK_CALLERS_SLOT, idx), InvalidCallbackCaller());
require(scriptHash == expected, InvalidCallback());
callbackIdx = idx + 1;
```

- `msg.sender` is still `FlashLoanHelper`, which matches the pre-committed callback caller
- `scriptHash` matches the approved inner callback hash `H1`
- `callbackIdx` advances, so the callback is marked as consumed

7. Control returns to the controlled contract, which sends the original flash-loaned funds back to `FlashLoanHelper` for repayment. The outer call frame then finishes normally, and `OnchainPM.execute()` observes that all callbacks were consumed.

```
require(callbackIdx == TransientArrayLib.length(CALLBACK_HASHES_SLOT), UnconsumedCallbacks());
```

The result is a clean end-to-end transaction:

- the strategist had arbitrary use of the full flash-loaned capital outside the VM
- the approved inner callback still ran on the real OnchainPM
- Aave was fully repaid
- `execute()` completed without reverting
- on-chain state only shows a normal callback consumption path

Note that `callbackData` (the `bytes calldata` parameter forwarded through Aave as `params`) is itself likely an unpinned state slot in the outer script as it varies per pool just as `onchainPM` likely will in such multi-pool management scenarios. This means they control both the routing target and the inner script parameters from a single script with two variable slots, giving full control over the `(commands, state, stateBitmap)` tuple decoded in `executeOperation`. The strategist can craft `callbackData2` to ensure the inner script succeeds regardless of token balances.

This eliminates all three defenses.

Recommended Mitigation:

1. Add a reentrancy guard to `requestFlashLoan` to block the malicious pool re entry path.
2. Require that the `onchainPM` that will receive the callback is the `msg.sender` of the `requestFlashLoan` call.
3. Validate `onchainPM` is a legitimate factory-deployed instance:

```
contract FlashLoanHelper is IFlashLoanHelper, IAaveV3FlashLoanReceiver {
   using SafeTransferLib for address;

   IOnchainPMFactory public immutable factory;

   address private transient _pool;
   address private transient _onchainPM;

   constructor(IOnchainPMFactory factory_) {
      factory = factory_;
   }

   function requestFlashLoan(
      IAaveV3Pool pool,
      address token,
      uint256 amount,
      IOnchainPM onchainPM,
      bytes calldata callbackData
   ) external {
      require(_pool == address(0), AlreadyActive());  // reentrancy guard
      require(msg.sender == onchainPM); // caller can only request a callback to itself
      require(
         factory.getAddress(onchainPM.poolId()) == address(onchainPM),
         InvalidOnchainPM()
      );
      _pool = address(pool);
      _onchainPM = address(onchainPM);
      emit FlashLoan(address(pool), token, amount, address(onchainPM));
      pool.flashLoanSimple(address(this), token, amount, callbackData, 0);
      _pool = address(0);
      _onchainPM = address(0);
   }
   // ...
}
```

Note that hardening the contract with these checks can also prevent other potential attacks that share similar mechanism. So it's recommended to implement them even if there are other factors or potential mitigations that defeat this specific attack scenario.

Client: Fixed in centrifuge/protocol-internal#175.

BurraSec: Fix looks good. Recommendation implemented.

7.2.5 SlippageGuard cumulative period loss and per-trade maxSlippageBps can be bypassed with deposits by other parties

Target: SlippageGuard.sol

Severity:

- Impact: Medium
- Likelihood: Low

Description: The deposit/withdraw tracking mechanism in `SlippageGuard` (cumulative and per trade), only capture the total difference in available balance between two moments (at `open` and at `close`).

```
(uint256 withdrawn, uint256 deposited) = _computeDeltas(poolId, scId, poolDecimals);
if (withdrawn > 0) {
   uint256 loss = withdrawn > deposited ? withdrawn - deposited : 0;
   require(loss <= withdrawn * maxSlippageBps / 10_000, SlippageExceeded(withdrawn, deposited,
   ↪  maxSlippageBps));

   if (loss > 0) {
      _trackPeriodLoss(poolId, scId, loss.toUint128());
   }
}
```

`_computeDeltas()` at `SlippageGuard.sol:118-147` reads `balanceSheet.availableBalanceOf()` for each tracked asset. This reflects all balance changes regardless of what originated it.

The problem is that there might be operations that are not part of the script execution that happen during the execution. For example, an `OnOffRamp.deposit` changes the return value of `balanceSheet.availableBalanceOf()`.

This requires an arbitrary contract to be call to be triggered from within the script execution, or OnOffRamp actions to be deliberately added to the script. While well defined scripts that target only well behaved protocols and assets could avoid this risk, it's hard to guarantee it won't happen in the future, in particular as more assets, script templates, and protocol interactions are added.

The impact would be allowing the strategist to produce a loss in excess of the set parameters, potentially keeping the funds for themselves.

Recommended Mitigation: A mechanism should exist to prevent withdrawals/deposits/notes to the `BalanceSheet` from being triggered by other contracts during script execution. For example, the `BalanceSheet` could have a mechanism to temporarily lock it to accept calls only from the `OnchainPM` using a transient storage lock. As the typical use case seems to be to call those operations directly from the `OnchainPM` during script execution, this would effectively prevent any violation of the expected scenario.

Client: This is ultimately a part (and indeed a limitation) of the design

BurraSec: Design limitation with unfeasible fix.

7.2.6 ScriptHelpers: decimal conversion has fixed rounding down behavior.

Target: ScriptHelpers.sol

Severity:

- Impact: Medium
- Likelihood: Low

Description: `scaleDecimals` in `ScriptHelper` always rounds down when reducing to lower decimals count. This might lead to unintended rounding down that benefits the strategist (or third parties) at the expense of the protocol.

Recommended Mitigation: Add a parameter to define the direction of rounding in case the number of decimals is reduced.

Client: Fixed in centrifuge/protocol-internal#175.

BurraSec: Fix looks good. Rounding parameter added.

7.3 Informational

7.3.1 SlippageGuard can be DoS'd by other calls in same transaction

Target: SlippageGuard.sol

Severity: Informational.

Description: The function "open" in SlippageGuard can be called by any sender, setting the asset list in transient storage, which locks the contract until the same msg.sender calls "close". While transient storage is wiped out at the end of each transaction, any contract that is called within the same transaction as the script execution could force the legitimate slippage guard "open" operation from the OnchainPM to revert.

There are two potential scenarios for this DoS:

1 - Any external contract call that is triggered within the script but before `OnchainPM` calls `SlippageGuard.open` might potentially front run with it's own call to `SlippageGuard.open` before and cause the legitimate operation to revert.

2 - A strategist that is a safe or some other complex contract that bundles external calls before even starting the script execution, potentially allowing for this attack vector to be used.

Recommended Mitigation: The within-script-execution DoS scenario can be avoided by having `SlippageGuard.open` be the first operation in a script. The second scenario, with the DoS being triggered before the script execution, can be avoided with careful strategist design. These caveats should be documented.

Client: Fixed in centrifuge/protocol-internal#175.

BurraSec: Fix looks good. Slippage guard now has onlyOnchainPM modifier.

7.3.2 Lack of callback+command ordering guarantee in OnchainPM

Target: OnchainPM.sol

Severity: Informational

Description: The callback mechanism only validates the ordering within a sequence of callbacks -- it can't validate which of the commands (or callbacks) triggered the callback execution. This can allow a situation where a command unexpectedly triggers a callback while the later command expected to trigger it doesn't (or the opposite), leading to state changes not anticipated by the script masks etc. In particular there are missing validations in `FlashLoanHelper` that allows for this sort of behavior to be exploited.

Example:

Expected flow:

1. Command 1 is executed.
2. Command 2 is executed.
3. Callback 1 is executed.

Actual flow:

1. Command 1 is executed.
2. Callback 1 is executed.
3. Command 2 is executed.

Both flows are compatible with the same script hash as the ordering of callbacks is tracked independently from the ordering of top level commands.

Exploitation would require the script execution to trigger a call to a strategist controlled contract, which is most likely limited to only malicious assets or assets with hooks -- a serious limitation on the feasibility of the attack. However, the possibility can't completely be ruled out, in particular as more assets are added in the future.

Recommended Mitigation: OnchainPM should keep track of the index of the (top level) command is being executed so the callback can be validated to match it. Besides the callback data and caller, the `execute` function should also receive as arguments the index of the top level operation after which each callback is expected to happen (alternatively callbacks could be included in the sequence of commands with a flag). During `executeCallback` the current operation index should be validated.

Client: Fixed in centrifuge/protocol-internal#175

BurraSec: Fix looks good. Limitation documented.

7.3.3 6909 transferFrom reduces allowance when approval is set to max

Target: AccountingToken.sol

Severity: Info

Description: AccountingToken transferFrom subtracts the transfer amount from allowance after the sufficiency check whenever msg.sender differs from the sender. Approving the maximum uint256 value therefore does not implement infinite approval semantics where a allowance set to max does not decrease when spent. Instead each transferFrom still decreases stored allowance. Potentially impacting integrators that expect max allowance to read as maximum uint256 at all times.

Recommended Mitigation: Document that allowance decreases on every transferFrom and max uint is a large finite budget not an infinite flag.

Client: Fixed in centrifuge/protocol-internal#175.

BurraSec: Fix looks good. Natspec updated.

7.3.4 Admin paths emit after no-op storage write

Target: AccountingToken.sol, OracleValuation.sol

Severity: Info

Description: trustedCall-style admin paths write mapping values from decoded payloads, then emit events with the new tuple. When the new value equals the stored value, storage is unchanged but the event still fires. 3rd party users that treat every emission as a configuration change over-count churn. The `AccountingToken::trustedCall` function sets minters and emits `UpdateMinter` with the decoded flag even when the minter already held that boolean. The `OracleValuation::updateFeeder` function sets `feeder` and emits `UpdateFeeder` with `canFeed` even when that feeder already held the same permission.

Recommended Mitigation: Emit only when storage differs from the payload, or document that these events report final state not deltas so indexers read storage after the log.

Client: We emit events like this in most places in the codebase, so will leave as is.

BurraSec: Acknowledged

7.3.5 ScriptHelpers: `abs` unexpectedly revert on minimum valid signed integer.

Target: ERC20.sol, ERC721.sol, etc.

Severity: Info

Description: The minimum valid value for a int256 can't be converted to a positive int256 using the expression `-x` as it will overflow the `int256` type. However, the `uint256` type can represent the magnitude of that value correctly without overflowing. By using the expression "-x" the function `abs` reverts unexpectedly when the valid minimum value is used in the argument.

Recommended Mitigation: While the minimum value is unlikely to happen in real usage, the lack of support should be at least documented. Ideally the implementation should be changed to support it. E.g. use two's complement.

Client: Fixed in centrifuge/protocol-internal#175.

BurraSec: Fix looks good. Now handles full range.

7.3.6 ScriptHelpers: addBps unnecessarily limits addition to 100%.

Target: ScriptHelpers.

Severity: Info

Description: The function `addBps` reverts when the percentage used is > 100%. There's no mathematical reason a bps addition shouldn't be able to add more than 100% to a value. While this is unlikely to happen in any normal usage, it seems an unnecessary restriction (and also a small amount of extra gas).

Recommended Mitigation: Remove the require statement that prevents adding more than 100%.

Client: Fixed in centrifuge/protocol-internal#175.

BurraSec: Fix looks good. Cap removed.

7.3.7 Lack of validation of non-empty assets array in SlippageGuard

Target: SlippageGuard.sol

Severity: Info

Description: SlippageGuard doesn't validate in `open` that the provided asset list is non-empty. Since the array length is used as a reentrancy guard, it can lead to the invalid state that the caller has been set but "open" can be called again before "close" is called. Won't cause any impact, but is inconsistent.

Recommended Mitigation: The `open` function should verify that the asset list is non-empty.

Client: Fixed in centrifuge/protocol-internal#175.

BurraSec: Fix looks good. Non zero asset list ensured.

7.3.8 State bitmap isn't checked to contain only bits for valid state indexes

Target: OnchainPM.sol

Severity: Info

Description: State hashing (function in `OnchainPM`) accepts as argument bitmaps with ones for indexes that dont exist in state array. While this is doesn't affect execution, since the bitmap itself is hashed and validated agains the Merkle root. However the presence of such bits would indicate the script contains invalid data and might not be correct.

Recommended Mitigation: should assert/require that remaining bits after count is zero.

Client: Fixed in centrifuge/protocol-internal#175.

BurraSec: Fix looks good. Require condition was added

7.3.9 Lack of index range validation in TransientArrayLib

Target: TransientArrayLib.sol

Severity: Info

Description: The newly introduced functions `at` and `atAddress` don't validate that the index is within range, allowing the caller to access data potentially corrupted or set in a previous execution (within the same transaction).

While no present use of the function is vulnerable, the broken expectation (Solidity checks indexes by default) makes the library likely to be misused in the future.

Recommended Mitigation: Add `require` statement that the index is less than the length of the array.

Client: Fixed in centrifuge/protocol-internal#175.

BurraSec: Fix looks good. Bounds check added.